

FastAPI Document Service Workflow

End-to-End PDF Upload, Chunking, and Vector Storage Flow

1. User uploads a PDF to POST /documents/upload.
2. FastAPI matches the request to `upload_document()`.
3. `verify_token()` runs first via `Depends()`.
4. The uploaded file becomes an `UploadFile` object.
5. The filename is validated to ensure it ends with `.pdf`.
6. `process_pdf(file)` is called.
7. `await file.read()` loads the PDF bytes into memory.
8. PyMuPDF opens the PDF using `fitz.open(stream=content, filetype='pdf')`.
9. Each page's text is extracted using `get_text()`.
10. `RecursiveCharacterTextSplitter` splits the text into chunks.
11. Chunks, metadata, and IDs are collected.
12. `add_texts()` sends chunks to the vector store.
13. Embeddings are generated for each chunk.
14. Embeddings and metadata are stored in ChromaDB/Pinecone.
15. A success message is returned to the API caller.

Recursive Chunking Explained

RecursiveCharacterTextSplitter tries to split text intelligently. It first tries paragraph breaks (`\n\n`), then line breaks (`\n`), then spaces, and finally individual characters if necessary. The goal is to preserve meaning while keeping chunks under the configured `chunk_size`.

Example:

Original text -> Paragraphs -> Lines -> Words -> Characters (last resort).

Only the final chunks are stored in the vector database. The recursion process itself is not stored.

Data Stored in the Vector Database

Example Chunk:

'Python is a programming language.'

Metadata:

{'source':'book.pdf','page':1,'chunk':0}

ID:

book.pdf-p1-c0

The vector store generates an embedding vector for each chunk and stores: chunk text, metadata, ID, and embedding.